



MASTER'S THESIS
(COURSE CODE: XM_0011)

Evaluation of Runahead Execution on the NVIDIA Denver 2 CPU

by

Yuxuan Chen

(STUDENT NUMBER: 2729367)

*Submitted in partial fulfillment of the requirements
for the degree of
'Computer Security' Master of Science
in
Computer Science
at the
Vrije Universiteit Amsterdam*

August 21, 2025

Certified by

Cristiano Giuffrida
Full Professor
First Supervisor

Certified by

Mathé Hertogh
PHD Candidate
Daily Supervisor

Certified by

Herbert Bos
Full Professor
Second Reader

Evaluation of Runahead Execution on the NVIDIA Denver 2 CPU

Yuxuan Chen

Vrije Universiteit Amsterdam

Amsterdam, NL

y30.chen@student.vu.nl

ABSTRACT

Runahead is a specific type of speculative execution designed to enhance memory-level parallelism. During the runahead period, the CPU pre-executes instructions to detect potential cache-miss loads and issues prefetch requests to bring the corresponding data from main memory into the data cache in advance [10, 31]. Despite its performance benefits, runahead execution can expose systems to security risks, particularly Spectre-like attacks [38]. In this paper, we present a comprehensive evaluation of runahead execution in the NVIDIA Denver architecture.

NVIDIA Denver employs dynamic code optimization (DCO), acting as a just-in-time compiler to reorder and modify instructions, thereby replacing the complex reorder buffer [3, 4]. Since DCO complicates the CPU reverse engineering analysis, we first provide a systematic analysis of DCO and present strategies to avoid triggering it. Additionally, we present a fingerprint method to distinguish runahead execution from conventional speculative execution, ensuring that the observed instruction window expansion is attributed solely to runahead and not to speculative execution. Furthermore, we analyze runahead at the ISA level and present the results of whether a specific ARM instruction executes as expected in runahead. We also observe that runahead introduces new security threats to in-order CPUs because runahead efficiently manages the instruction dependency during the pre-execution period. Finally, we evaluate runahead in terms of security and investigate multiple Spectre attacks via runahead.

1 INTRODUCTION

Modern CPUs employ multiple optimizations to enhance performance. Techniques such as out-of-order execution [27, 41] and superscalar [40] focus on improving instruction-level parallelism performance to increase throughput. However, complicated CPU optimizations also introduce security risks. For example, several Intel CPUs allow memory permission checks and memory access to run in parallel, which introduces the Meltdown vulnerability [28]. Spectre-type attacks are well known for their efficiency in exploiting the CPU’s predictive behaviors across different components [25]. Microarchitectural attacks can exploit multiple targets such as cryptographic processes [6], Intel SGX environment [42], KASLR [20], Chrome isolation [2], etc.

In contrast, in-order superscalar CPUs are commonly regarded as immune to Spectre attacks because they strictly issue instructions in order. While some research suggests that an in-order CPU, such as the ARM Cortex-A53, is vulnerable to the Spectre attack, the CPU does not efficiently manage instruction dependencies within the speculative execution window. Consequently, the outcomes of all instructions executed speculatively cannot be retrieved by following instructions, making Spectre attacks inefficient on these CPUs [33].

Runahead is a type of speculative execution that has garnered attention for its performance benefits. Although much of the research focuses on these performance aspects, one study highlighted potential security issues associated with runahead by simulating it in the Sim2Multi simulator [38]. Although evaluating runahead in simulated environments exposes numerous potential security issues, few studies investigate runahead in a black-box and real-world context. Given that NVIDIA Denver commercially implements runahead [3, 4], our aim is to use the Denver CPU as a target to explore the behaviors of runahead in a non-simulated setting.

The NVIDIA Denver CPU utilizes runahead technology as an advanced prefetcher that follows the program flow to reduce the overhead associated with off-core data requests [3, 4]. Runahead exhibits similar behavior to speculative execution, and it could make in-order superscalar CPUs vulnerable to transient attacks. In this paper, we investigate whether runahead execution introduces more security risks and whether it can expose in-order superscalar architectures to transient attacks by reverse engineering the Nvidia Denver architecture.

We conducted black-box experiments on the Denver to investigate runahead execution. Specifically, we primarily employ a flush-reload side-channel [46] and the Spectre-v1 attack to explore the behavior of runahead. A key challenge is that runahead is similar to lightweight speculation on in-order CPUs, making it difficult to distinguish between them. To resolve this, we fingerprinted runahead using its unique behavior of skipping floating-point/vector instructions. Additionally, we analyze runahead at the ISA level, which indicates that apart from floating-point/vector operations, the data memory barrier is also skipped, and accessing system registers in user space is not permitted. Additionally, runahead may utilize pseudo-retirement to manage complex data dependencies and efficiently handle the temporal results of all instructions. Shifting to a security perspective, we tested the Spectre-v1, Spectre-SSB, and Spectre-RSB attacks, as well as Meltdown. We observe that all three Spectre variants work in the context of runahead, but Meltdown does not. During our security analysis, we identified a new variant of the Spectre attack, Spectre-FPV-bypass, that arises from the feature that the CPU skips FP/vector operations in runahead.

Our contributions are summarized as follows:

- We provide a specific method for distinguishing between runahead and speculative execution, as well as a proper technique for fingerprinting runahead.
- We investigate whether runahead increases the risk of in-order superscalar CPUs being affected by Spectre attacks and explain why runahead makes Denver CPUs more susceptible to such attacks.
- We conduct tests on various Spectre attacks using runahead to demonstrate that most Spectre attacks are effective on CPUs that implement runahead.

- We introduce a new variant of Spectre attacks by leveraging the unique feature of runahead.

2 BACKGROUND

2.1 Runahead execution

Long-latency operations, such as cache misses, typically activate Runahead. During this period, the processor pre-executes instructions to identify potential load operations that may result in a cache miss, rather than waiting solely for the data to become available. When a cache miss occurs, the CPU generates a prefetch request to retrieve the required data from the main memory and store it in the cache [10, 31]. Runahead utilizes the CPU-stalling period to prefetch data, thereby warming the cache and resulting in a lower cache miss rate in subsequent executions.

2.1.1 Propagation of invalid data. In runahead execution, the flow of operations differs from that in standard execution because the CPU bypasses many instructions due to the unavailability of operands and conditions. For example, if particular instructions depend on a previously cache-missed load, those instructions will be invalidated and skipped. As illustrated in Figure 1, suppose the instruction 1 is a cache-missed load, which triggers runahead execution; however, the required data is not available at the moment; consequently, the execution unit will invalidate register x0. Additionally, instructions 2 and 3 both depend on the value of x0, so the CPU will invalidate these two instructions because the dependent operands are not available. The instructions without dependencies on prior invalid instructions are allowed to execute, such as instruction 4 and 5. Instruction 4 is a load operation; it first accesses the data cache to retrieve data. If the access hits data in the cache successfully, destination register x3 will be updated, and the subsequent instruction 5 will also be valid, updating register x4. However, if the load operation results in a cache miss, as in instruction 6, it will not execute. Instead, the core will prefetch for the data, and the instruction 7, which depends on the previous load operation, will be invalidated. Instruction 8 is special in runahead; the store operation will not write data to the data cache nor DRAM. Instead, it will place the temporary data into a cache, such as a runahead cache or store buffer; consequently, instructions that depend on store operations can retrieve data from the runahead cache.

2.1.2 Microarchitectural execution vs architectural execution. Architectural execution refers to the updates of the machine state that are visible to the programmer, such as changes to registers and memory. It does not involve the details of how instructions are executed internally by the processor. For example, an add instruction, at the microarchitectural level, involves several steps: the instruction is first fetched from the cache into the processor, decoded by the decoder, and generates corresponding operation signals. The control logic then directs the relevant register values to the Arithmetic Logic Unit (ALU) for computation. Finally, the CPU updates the result in the register file or memory and retires the instruction. However, from the programmer's perspective, only the final update to the register file is observable, and the intermediate microarchitectural processes remain hidden [17].

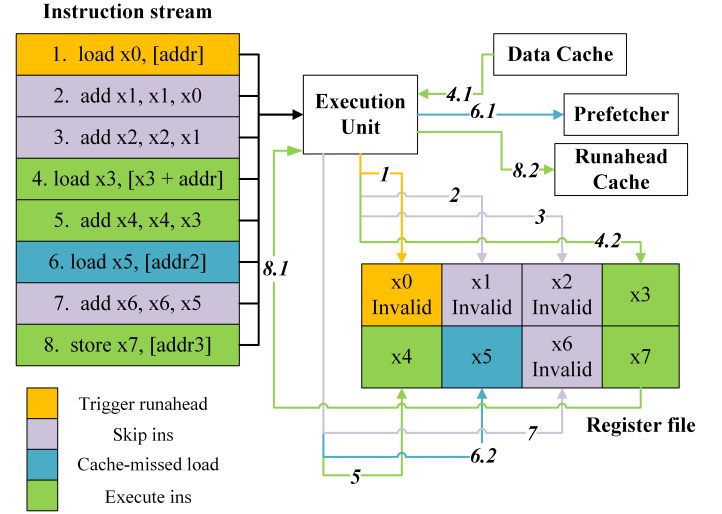


Figure 1: Runahead execution

- 1 – Trigger runahead, invalidate x0.
- 2 – Skip instructions that depend on 1.
- 3 – Skip instructions that depend on 2.
- 4 – Access data located in data cache(4.1). Update x3(4.2).
- 5 – Update x4.
- 6 – Discover cache-missed data, send prefetch request (6.1). Invalidate x5 (6.2).
- 7 – Skip instructions that depend on 6.
- 8 – Retrieve data from x7(8.1). Place data in runahead cache(8.2).

2.1.3 In-order execution vs out-of-order execution. The difference between in-order and out-of-order execution is the order in which instructions execute. In-order CPUs issue instructions strictly in program order. By contrast, out-of-order CPUs enable the scheduler to issue instructions when operands are available and utilize a re-order buffer to maintain program order at commit. NVIDIA Denver is an in-order CPU with runahead, which pre-executes instructions in order under a cache miss to uncover further misses. While in runahead mode, the scheduler continues issuing instructions in order. Our observations indicate that Denver lacks a reorder buffer and issues all instructions strictly in order, both during normal execution and runahead. On the other hand, out-of-order CPUs can issue instructions out of order at all times.

2.1.4 Runahead vs speculative execution. Runahead execution and speculative execution mainly differ in their purposes. Runahead is a microarchitectural optimization designed to improve memory-level parallelism by prefetching data during long-latency stalls, especially cache misses. It does not update the architectural state, as all results during runahead execution are discarded, with only prefetched data retained in the cache [36]. In contrast, speculative execution aims to enhance instruction-level parallelism performance by predicting future execution flows, such as branch prediction [39] and value prediction [13]. If the prediction is correct, the results will be committed to the architectural state as the memory system and register file; otherwise, the CPU flushes the pipeline and recovers the state [12].

2.1.5 Runahead vs prefetching. Prefetching and runahead execution both aim to mitigate memory access overhead, but they exhibit

different behaviors. Prefetching relies on the statistical analysis of memory access patterns to predict future data accesses, often using hardware or software-based algorithms [43]. In contrast, runahead execution speculatively pre-executes instructions beyond the stalled point. Although runahead also triggers prefetch requests, it follows the program’s actual control flow rather than relying solely on predicted patterns.

2.2 Architecture of NVIDIA Denver

Denver is designed by NVIDIA, based on the ARMv8 ISA. The first generation of Denver is superscalar but in order. Figure 2 illustrates the overall architecture of Denver, which features two ARM decoders and one optimized decoder that incorporates a dynamic code optimizer. The discussion of this optimizer is covered in section 5. The backend can execute up to seven uops simultaneously as a bundle, which includes two load/store units, two ALUs, two floating-point/neon operation units, and one branch unit. Additionally, Denver is equipped with a branch predictor that fetches instructions speculatively to reduce overhead. Alongside the branch predictor, Denver also features a return buffer to accelerate the process of returning from a function call.

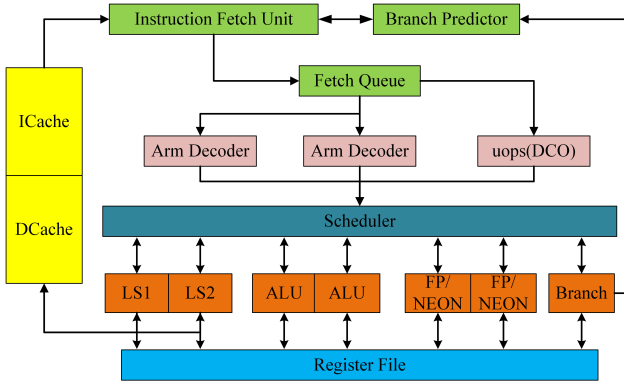


Figure 2: Nvidia Denver CPU

2.2.1 Branch predictor. Most modern CPUs employ a branch predictor to fetch instructions speculatively in advance, rather than waiting for the branch to be resolved in the branch unit. Out-of-order CPUs can execute instructions speculatively, based on the outcome of the branch predictor. The branch predictor records whether a branch is taken or not and decides whether to take the branch based on statistical analysis of the recorded history. If the prediction is incorrect, the CPU will flush the pipeline and clean the results in the speculative path [39].

2.2.2 Return stack buffer. The return stack buffer stores the return address of a function call and is close to the core; thus, the latency of accessing the return buffer is significantly lower than accessing the memory system. When the CPU calls a function, it automatically pushes the return address into the return stack buffer and the memory stack. When returning from the function, the CPU follows the top return address in the return stack buffer speculatively. However, if the prediction is incorrect, the CPU will also flush the pipeline and revert to the correct execution path [26].

2.3 Dynamic code optimization

NVIDIA’s Denver CPU includes a dynamic code optimizer (DCO), which serves as a software-based binary translator designed to improve execution performance by analyzing and optimizing microcode during execution [4]. The overall workflow of the DCO system is illustrated in Figure 3. The branch unit monitors the execution frequency of the basic blocks. When the execution count of a block exceeds a predefined threshold, the CPU forwards the block to the DCO. The optimizer then analyzes the microcode and translates the microcode into an optimized version and directly propagates this optimized code to the execution units in order to improve the instruction-level parallelism performance [11].

However, this form of runtime optimization introduces challenges in analyzing the CPU’s behavior. By dynamically altering code execution, DCO complicates both program debugging and the study of processor behavior, as described in previous work [18]. The DCO system performs several advanced optimizations, including loop unrolling, loop-invariant loading lifting, reordering of loads and stores, and removing redundant instructions, among others [3, 4].

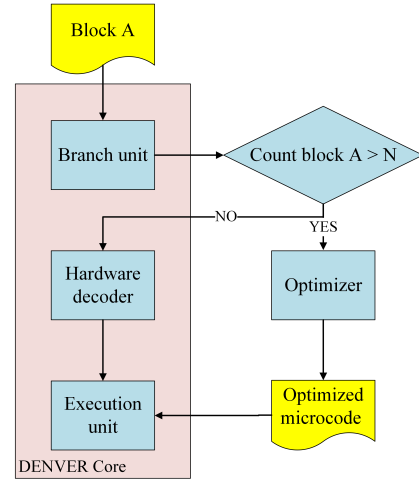


Figure 3: Dynamic code optimizer in NVIDIA Denver

3 THREAT MODEL

We assume a local attacker capable of executing arbitrary code in user space, i.e., at Exception Level 0 (EL0). The system runs on an NVIDIA Denver CPU, which supports runahead execution. The attacker has no privileged access and cannot directly read sensitive information, such as kernel memory or private data from other processes. However, the attacker can launch microarchitectural side-channel attacks to infer secrets by exploiting any security issues introduced by runahead execution. Specifically, we assume the attacker can mistrain the branch predictor to influence control flow, search for Spectre-type gadgets in the system, and use precise performance counters to mount Flush+Reload attacks. The attacker’s goal is to leak data that is architecturally inaccessible, such as user-space secrets from applications or kernel memory. We

assume the attacker cannot gain root privileges, turn off security mitigations, or physically access the system.

4 OVERVIEW

First, we provide a behavior analysis of the DCO, which acts as a runtime binary translator and significantly complicates microarchitectural analysis. In Section 5, we introduce methods to detect the presence of DCO, investigate different phases of DCO, and propose techniques—such as loop unrolling and address randomization—to prevent DCO from interfering with our reverse engineering experiments.

Second, we explore the internal behavior of runahead execution on the Denver 2 CPU in section 6.2. We describe how to open a runahead instruction window, distinguish it from traditional speculative execution using a fingerprinting method in section 6.3, and analyze the execution behavior of various ARMv8 instructions within the runahead window in section 6.4. We further investigate the concept of pseudo-retirement and demonstrate that runahead can preserve a large number of instruction results even without a reorder buffer in section 6.5.

Finally, we conduct a thorough security analysis of runahead execution in section 7. In the evaluation section 8, we examine the effectiveness of multiple Spectre-type attacks, including Spectre-v1, Spectre-SSB, and Spectre-RSB, when combined with runahead. We also propose Spectre-FPV-bypass that leverages the skipping behavior of FP/vector operations in runahead mode, and assess its impact on real-world security.

5 DCO ANALYSIS

Since the DCO modifies the original microcode design during its optimization process, it introduces significant challenges to reverse-engineering the processor behavior. Given that the specific implementation details of the DCO are not publicly available, and extracting the underlying microcode from hardware such as the Jetson board is infeasible, we instead focus on analyzing the observable behaviors of the DCO, which allows us to study its impact indirectly.

5.1 Overview

We start by monitoring whether DCO is activated by measuring the latency of redundant instructions. Because DCO eliminates redundant operations, we expect to see a significant decrease in latency. Our observations reveal that latency can generally be divided into three stages 5.2. The first stage is the period during which DCO is not yet activated, and the CPU relies on the ARM decoder for instruction execution. In the second stage, the latency increases slightly; we believe this behavior occurs because the optimizer is analyzing the code during this period. In the third stage, the latency drops rapidly and remains constant, indicating the period during which the CPU is executing optimized microcode. Additionally, we observe that DCO can reuse the optimized code if the source code remains unchanged or is not recompiled. We also propose a countermeasure to address this issue.

5.2 Monitor whether the DCO is activated

As discussed in the previous section, the DCO may eliminate unused instructions. Using this feature, we can design an experiment to observe and verify whether the DCO is triggered.

5.2.1 Experiment design and hypothesis. We insert redundant instructions that are likely to be recognized by DCO as unused code and then measure the latency when executing these redundant instructions. By analyzing the latency, we can infer whether the DCO is actively optimizing the code and removing these redundant instructions. As demonstrated in algorithm 1, the experiment involves measuring the latency of a bundle of redundant instructions in multiple iterations. For this experiment, we used the `mov x0, x0` instruction as an unused instruction that does not affect the architectural state. In the experiments, we measured the latency of 100 repeated redundant instructions over 10,000 iterations to observe the behavior of the DCO.

In this approach, we hypothesize that if the DCO identifies specific instructions as redundant, it will remove them from the instruction bundle; as a result, we expect to observe a significant drop in latency after multiple iterations, indicating that the DCO has eliminated redundant instructions and the CPU is executing optimized microcode directly. By monitoring this behavior over multiple iterations, we can conclude that the DCO has completed the optimization process, as the observed latency will decrease once the redundant operations are discarded.

Algorithm 1 Measure redundant instructions in a loop

```

 $x \leftarrow N$ 
while  $x \neq 0$  do
     $time \leftarrow counter$  ▷ Read performance counter
     $repeat\_100(mov\ x0, x0)$ 
     $time \leftarrow counter - time$ 
     $x \leftarrow x - 1$ 
end while

```

5.2.2 Result and analysis. As shown in Figure 1, a significant latency drop is located between the 5,000th and 6,000th iterations. The latency approximately decreases from 180 to 50 cycles and stabilizes, indicating that the CPU has entered optimized mode. The drop of around 110 cycles corresponds closely to the entire latency of redundant instructions.

The observed latency reduction suggests that the DCO identifies the repeated instructions as redundant, optimizes the execution by removing them, and subsequently generates microcode that significantly reduces execution time. This behavior highlights the DCO's role in enhancing program efficiency by eliminating unnecessary instructions and dynamically adjusting the code layout based on execution patterns.

5.3 Three phases in DCO

In addition to the sudden drop in latency, the latency exhibits three distinct phases. We hypothesize that, during the first phase, the DCO has not yet kicked in. In the second phase, DCO is actively analyzing the code, while in the third phase, the CPU executes the optimized microcode generated by DCO. We also observed that in

Phase 3, there are no further changes, regardless of the number of mov instructions. We believe that Phase 3 represents the stage after DCO optimization, where the processor no longer uses the ARM decoder but executes the optimized microcode directly from the translator cache.

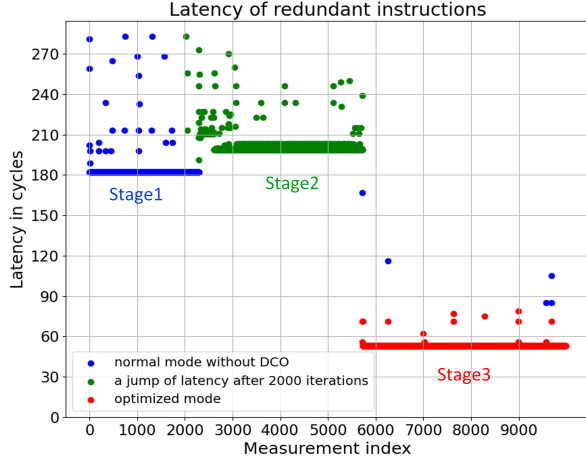


Figure 4: Latency measurement for redundant instructions

5.4 DCO behavior analysis phase 1

In the experiment observing the triggering of the DCO, we identified three distinct phases in latency measurements for redundant instructions 1: latency stabilizes at 180 cycles, a slight increase occurs at 210 cycles, and finally, a significant drop to 50 cycles. We consistently observed these three stages over multiple experimental repetitions.

We believe that in the first phase, latency remains stable because the processor operates in normal mode and has not yet reached the threshold to activate DCO. The number of loop iterations in this phase approximately corresponds to the threshold required to trigger DCO. Moreover, the duration of this phase remains constant regardless of variations in the number of instructions within the basic block.

5.4.1 Experiment design and hypothesis. We continue to measure the latency for 100, 200, 300, and 400 mov instructions; each experimental group repeats 2,000 times. We extract the initial region corresponding to the first phase before the latency rise occurs and record the number of iterations that this phase lasts.

Suppose the number of iterations in the first phase is similar across these four groups. In that case, it suggests that the previously identified first phase corresponds to the latency of instructions executed through the ARM decoder. Moreover, the duration of this phase is approximately equal to the threshold required to trigger the DCO.

5.4.2 Results. Since the iteration count of phase 1 is not stable, we cannot definitively determine the exact threshold value. However,

as observed in Box Plot 5, the means and data distributions of the four experiments are approximately similar. We can conclude that during phase 1, the processor operates using the ARM decoder, and the threshold is approximately 2,100 iterations.

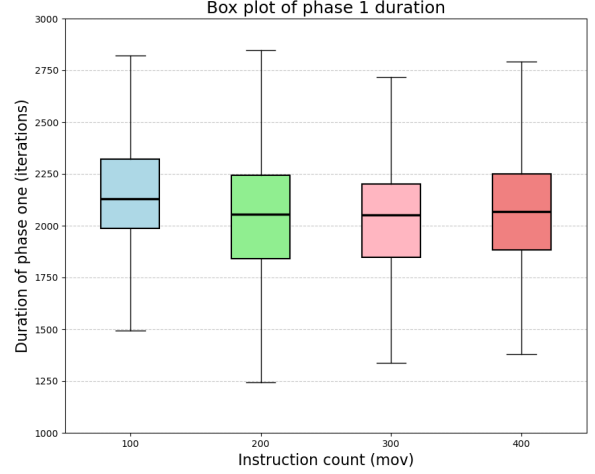


Figure 5: Box plot of four experiments recording the duration for phase one in iterations

5.5 DCO behavior analysis phase 2

In the second phase, the latency of the mov instruction block increases by approximately 10-15 cycles. We assume that the DCO analysis process is the cause of this delay. However, under idle system conditions, the DCO analysis phase should be running on another CPU core [3], and such delays should not occur. We suspect that the interactions between the core running the program and the DCO module may cause overhead, resulting in a slight increase in latency.

5.5.1 Experiment design and hypothesis. We continue to use redundant mov instructions and record the run-time latency for sets of 100, 200, 300, and 400 mov instructions. Subsequently, we extract the duration, in terms of loop iteration counts, for the section where the latency exhibits a slight increase. Since a greater number of instructions should require more time for the DCO to optimize, if the region of slightly increased latency is indeed related to DCO optimization, the duration of this region should progressively increase for 100, 200, 300, and 400 instructions.

Suppose we observe in the experiment that the delay slightly increases as the number of loop iterations increases for 100, 200, 300, and 400 mov instructions. In that case, we can infer that the region of slightly increased delay observed in previous experiments is related to the optimization of the DCO.

5.5.2 Result and discussion. From Figure 6, we observe that the number of loop iterations progressively increases as the number of mov instructions increases from 100 to 400. Although we cannot determine the exact value of this growth due to noise factors, the

data distribution from the four sets of results suggests that the increase is relatively stable. For every additional 100 mov instructions, the number of iterations for phase 2 consistently grows by approximately 600 to 1,000 iterations.

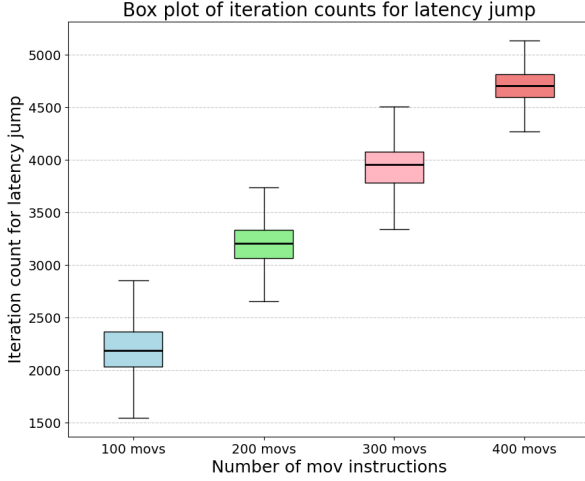


Figure 6: Recording the duration of the slight latency increase in terms of the number of loop iterations across four experiments.

However, according to the NVIDIA Denver documentation [3], DCO will run on an idle core. In theory, the core that executes the instructions should not experience a noticeable increase in delay.

The delay is likely caused by the processor performing a lookup in the translator cache for the corresponding optimized instructions. Specifically, when the number of instructions in a basic block exceeds a threshold, the DCO is triggered and starts optimizing the basic block. Before the final optimized microcode is generated, the processor will continue to perform a lookup in the translator cache at the beginning of each executing basic block to check if the corresponding optimized microcode is available [19].

5.6 Reusing Optimized Code in Subsequent Runs

In our experiments, we observed that the DCO may retain the optimized microcode in the cache. If a program undergoes optimization and is not recompiled or modified, it will enter the DCO mode more quickly upon subsequent executions.

In Figure 7, we can observe that the portion of the instructions decoded by the ARM decoder lasted for less than 1,000 iterations. After that, the latency for 100 instructions quickly decreased. Unlike in previous experiments, phase 2 is absent, meaning that when the original binary continues to execute, the processor does not go through the DCO analysis phase. Instead, it uses the microcode that already exists in the cache.

If the binary files are only deleted and recompiled before each run, there is still a significant chance that the processor will utilize the previously generated microcode. In the experiments we conducted,

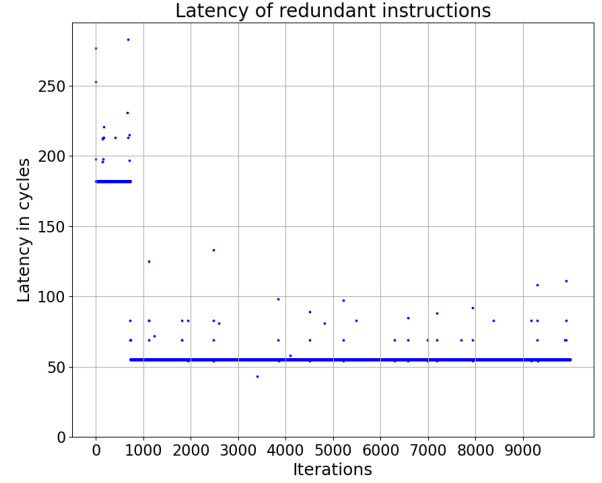


Figure 7: Optimized Code is reused in subsequent runs

about 150 out of 2000 runs showed that the latency for the mov instruction block dropped rapidly after fewer iterations. Therefore, simply recompiling the code does not fully resolve this issue.

5.7 Preventing the CPU from Entering Optimization Mode

We previously concluded that if a basic block contains more instructions, the time required for the DCO analysis will increase. However, the number of instructions does not affect the threshold for triggering the DCO analysis. However, we still need more effective methods to prevent code execution from entering the DCO-optimized phase; therefore, we need to reduce the number of loop iterations.

5.7.1 Loop unrolling. Loop unrolling is a technique that expands the loop body by replicating its operations multiple times, reducing the overhead associated with loop control. Loop unrolling modifies the termination condition to match the increase in the size of the iteration steps and minimizes or removes redundant branch instructions, improving execution efficiency [9].

In a general DCO system, the branch unit tracks the execution frequency of each basic block and uses the execution counts to determine whether a block requires optimization. However, suppose we employ loop unrolling in the program, the original basic block becomes larger to accommodate more instructions, which potentially extends the time required for DCO analysis. More importantly, with loop unrolling, we need fewer iterations to complete the task; consequently, the execution count of the basic block may drop below the threshold necessary for optimization.

5.7.2 Countermeasure for optimized microcode reusing. We prove that the optimized code could be reused frequently by DCO. To address this, we refer to research related to binary translation. One study indicates that when the number of executions for a basic block exceeds a threshold, the processor passes the address of this basic

block to the binary translator and then performs a lookup in the translator cache based on this address [19]. NVIDIA DCO might use a similar design. Therefore, we can ensure that the address of the basic block is different each time the program runs. We can achieve this by inserting a varying number of nop instructions before the basic block, ensuring that the number of nop instructions is different each time 2. In this way, the address of the basic block will be different in each run.

Algorithm 2 Measure Redundant Instructions with Preceding nops

```

M ← value      ▷ Change the value each time when compiling
nops(M)        ▷ A bundle with M nops
x ← N
while x ≠ 0 do
  time ← counter      ▷ Read performance counter
  repeat_100(mov x0, x0)
  time ← counter − time
  x ← x − 1
end while
  
```

5.8 Summary of DCO analysis

Through our experiments, we have identified some characteristics of DCO. To mitigate the challenges DCO may pose during the runahead reverse engineering process, we can employ loop unrolling to minimize excessive iterations within a loop. Additionally, by inserting a variable number of instructions each time and recompiling the binary, we can prevent DCO from utilizing the cached optimized code. Based on our findings, we believe that a loop with fewer than 800 iterations typically does not trigger DCO.

6 RUNAHEAD REVERSE ENGINEERING

Runahead reverse engineering experiments are conducted after a comprehensive analysis of DCO. All experiments described hereafter will be conducted by the following principle: they will circumvent the triggering of DCO. As discussed in the previous section 2.1, runahead exhibits similar features to speculative execution; thus, it is feasible to apply microarchitecture attacks, such as Spectre, to discover its characteristics.

6.1 Overview

We start by opening a microarchitectural instruction window using Spectre-v1 and page faults. However, it is essential to ensure that the instruction window is solely attributed to runahead and eliminates the possibility of speculative behaviors. We propose a fingerprint-based method to distinguish between runahead and speculative execution. Furthermore, we investigate the behavior of multiple standard ARM instructions during runahead and analyze the pseudo-retirement behavior in the runahead context.

6.2 Open runahead instruction window

Runahead is a form of microarchitectural optimization. To investigate the behaviors of runahead, it is essential to open a runahead instruction window and avoid architectural commitment. Triggering runahead is a cache miss. It is essential to make sure architectural execution does not follow the same code path as runahead.

```

1 /*
2  * Accessing array_length is delayed by cache miss.
3  * Branch is mistrained and mispredicted.
4  */
5 if(index < array_length){
6     // encode reloadbuffer
7     tmp = reloadbuffer[array[index] << 12];
8 }
  
```

Listing 1: Triggering runahead via branch misprediction in a Spectre-v1 gadget.

```

1 mprotect(buffer, 4096, READ_ONLY);
2 /*cache-missed load*/
3 load(&addr_miss);
4 /*illegal memory write, execute a bundle of NOPs*/
5 buffer[N] = 0;
6 NOPs();
7 /*encode the reloadbuffer*/
8 tmp = reloadbuffer[value << 12]
  
```

Listing 2: The second experiment try to trigger runahead from a page fault.

6.2.1 Instruction window: Spectre-v1. The Denver 2 CPU features a dynamic branch predictor and is superscalar [7], potentially exposing vulnerabilities such as Spectre-v1. Additionally, in runahead execution, if a branch instruction sources an invalid operand, the branch will not be resolved and will instead follow the outcome of the branch predictor [10]. As a result, the Denver 2 CPU could be susceptible to Spectre-v1 due to runahead execution.

Spectre-v1 is a type of Spectre attack which is efficient in exploiting conditional branches. In Spectre-v1, attackers mistrain the branch predictor, causing it to mispredict the direction of a branch. If there is a significant latency in resolving the branch, the processor will execute instructions along the mispredicted path [1, 24]. As presented in the list 1, if index is out of bounds of array, and the branch is mispredicted, data that should not be architecturally accessible may be encoded into reloadbuffer.

6.2.2 Instruction window: page fault. Another way to prevent architectural commitment is to halt execution with illegal operations, such as accessing invalid memory, which results in a page fault. Most CPUs follow a precise exception model, which allows exceptions to be generated precisely only after the older instruction has been retired [14]. As a result, some instructions can still be executed speculatively before the memory permission exception is handled, leading to a segmentation fault occurring in the end.

As shown in list 2, we can set up a read-only page using mprotect. After triggering runahead by accessing cache-missed data, we write to the read-only page to introduce a page fault. During this process, the CPU will continue to execute additional instructions before the exception is handled. If we open a runahead execution window successfully, we shall observe a cache hit in the reloadbuffer when we decode it within the signal handling function.

6.2.3 Results and discussions. Both experiments are successful. In Spectre-v1, we can see the cache hit signal outside the range of array, indicating that we can launch the Spectre-v1 attack with runahead execution. Additionally, when we introduce a page fault

in the runahead execution window, we also observe the cache hit signal in the reloadbuffer. These experiments demonstrate that a cache-missed memory access can trigger a runahead execution window, which is sufficient for further analysis.

However, we cannot conclusively state that the instruction window is introduced by runahead execution at this stage. Since the Denver 2 CPU is superscalar, even without runahead implemented, we can still observe the cache hit signal in the reloadbuffer. The ambiguity arises because runahead generally exhibits similar features to speculative execution. Therefore, it is crucial to design experiments that distinguish between runahead and speculative execution to confirm that the instruction window we have is genuinely due to runahead rather than other speculative mechanisms in a superscalar CPU.

6.3 Distinguish runahead execution and speculation

As we previously discussed, using a method like Spectre-v1 to open the instruction window does not allow us to confirm that it aligns with the runahead execution window. The challenge lies in distinguishing between runahead execution and speculative execution. In the runahead period, instructions from the deeper instruction stream, even those outside the reorder buffer, can also be executed [31], which leads to the conclusion that if we can measure the size of the reorder buffer [45], we can assert that the instructions executed outside of it are being executed in runahead mode because speculative execution is bounded by the reorder buffer, but runahead is not. However, Denver is an in-order architecture; thus, there may not be a reorder buffer in the CPU, and we do not observe any reorder buffer behaviors in the experiments.

6.3.1 Unique features of Runahead. Another way to distinguish between runahead and speculative execution is to identify a unique feature of runahead that is absent in standard speculative execution. In addition to basic behaviors such as pre-executing instructions, prefetching, and rolling back, several techniques have been proposed to enhance the performance and efficiency of runahead [16, 29, 30, 35].

6.3.2 Efficiency issue of runahead. A key performance issue with runahead execution is known as the useless runahead period. If the CPU enters runahead mode after a cache miss but encounters no load instructions during that period, it cannot generate effective prefetching [30]. To improve this, certain operations that do not contribute to memory address computation, like floating-point (FP) and vector operations, are skipped in runahead mode, allowing the CPU to execute more instructions and identify more potential cache misses. For instance, IBM Power 6 CPU skips FP operations during runahead mode [5].

6.3.3 Fingerprint runahead. To fingerprint runahead, we can modify the Spectre-v1 gadgets to introduce a data dependency between reloadbuffer encoding and FP operations 3. Initially, we open an instruction window using runahead as usual. Before encoding the reloadbuffer using `array[index]`, we introduce a data dependency between `index` and a FP register `d0`. Specifically, we initialize a FP register `d0` to 0 and convert its value to a normal arithmetic register `x0` by executing the `fcvtzs` instruction. Finally, we add `x0`

```

1 // array_length is cache-missed
2 if(index < array_length){
3     /*
4      *1. Initialize a floating-point register d0 to 0
5      *2. Covert floating-point value in d0 to x0
6      */
7     d0 = 0;
8     covert_float2int(d0, x0);
9     /*Introduce dependency between index and x0*/
10    index = index + x0;
11    tmp = reloadbuffer[array[index] << 12];
12 }

```

Listing 3: Experiment verifies FP ops are skipped in runahead.

to `index`. If FP operation `fcvtzs` is skipped, register `x0` should be marked as invalid, meaning that `index` will also be invalid. As a result, the encoding operation for `reloadbuffer` will be skipped because the value `array[index]` will not be retrieved due to the unresolved target address.

Initially, we establish a baseline by replacing the FP operations with an arithmetic operation `add`. In the experiment, we observe that the baseline gadget reliably encodes the `reloadbuffer`, indicating the correct execution of the gadget. In the experiment, when introducing the dependency between `index` and FP operations, we cannot observe the signal encoded in `reloadbuffer`. By comparing this behavior with the baseline, we confirm that the absence of signal is caused by skipping FP operations. However, it is essential to confirm that this behavior only occurs in runahead execution. In the next section, we will provide details that FP operations are not skipped during speculative execution.

6.3.4 FP operations in speculative execution. As a control group, we would like to verify whether FP operations are executed during speculative execution by placing them in the speculative window, while ensuring that runahead is not triggered.

Spectre-v1 relies on long-latency operations to delay branch resolution; however, a last-level cache miss triggers runahead. To overcome this challenge, we need to open an instruction window without causing a cache miss. In the Denver 2 architecture, the CPU can execute specific instructions out of order when it encounters a pipeline stall or contention for a backend resource. Since the Denver 2 CPU has only two units capable of handling load instructions simultaneously, performing three or more consecutive pointer dereferences within the branch instruction could lead to backend resource contention.

As illustrated in code snippet 4, instead of accessing a cache-missed `array_length`, we dereference a cache-hit three-level pointer `ptr3` to create a small instruction window. Subsequently, we execute FP operations followed by FP-dependent `reloadbuffer` encoding. If we observe the signal encoded in `reloadbuffer`, we can conclude that FP operations are not skipped in speculative execution.

In our experiment, we successfully executed Spectre-v1 by dereferencing a multi-level pointer, and we observed that floating-point instructions were indeed executed. Therefore, we can conclude that when the Denver 2 CPU experiences a cache miss, it enters runahead mode. In this mode, the instruction window is referred to

```

1  /*
2  * 1. Initialize three pointer
3  * 2. Link three pointers and array_length:
4  *   ptr3 -> ptr2 -> ptr1 -> array_length
5  */
6  size_t *ptr1, *ptr2, *ptr3;
7  ptr1 = &array_length;
8  ptr2 = &ptr1;
9  ptr3 = &ptr2;
10 if(index < ***ptr3){
11     d0 = 0;
12     covert_float2int(d0, x0);
13     index = index + x0;
14     tmp = reloadbuffer[array[index] << 12];
15 }

```

Listing 4: Control experiment to verify FP ops are executed as usual in speculative execution. There is no cache miss here; runahead is not triggered.

as the runahead window, during which floating-point operations are disabled.

6.3.5 Conclusion for fingerprint. We have successfully demonstrated that FP operations are skipped in runahead, but executed as expected in speculative execution. It indicates that the omission of FP operations is a feature specific to runahead and not a result of any pipeline hazards in speculative execution. Therefore, it is appropriate to use FP operations as a means of fingerprinting runahead.

Since both vector and FP operations are executed in the same execution unit, and given that vector operations typically do not contribute to address computation, it is reasonable to speculate that runahead similarly discards vector operations. Using the method as presented in list 3 and 4, we also verify that the CPU skips vector operations in runahead.

6.4 ISA-level behavior in runahead

As discussed in Section 6.3, we have triggered runahead and successfully fingerprinted it. It has already been demonstrated that FP and vector operations are skipped in runahead mode; however, we intend to investigate whether other types of instructions behave irregularly in this mode.

6.4.1 Arithmetic instructions. Arithmetic, logical, and bitwise instructions are evaluated by repeating the experiment presented in list 3, which introduces data dependencies between these operations and the variable index. For example, to test whether the add instruction executes correctly in runahead, we increment index by one before using `array[index]` to access the reloadbuffer¹. If the resulting cache-hit pattern shifts accordingly—indicating a one-position offset in the accessed memory—this implies that the add instruction was successfully executed during runahead.

6.4.2 Memory operations. We also tested load and store instructions to confirm whether memory access operations function as expected in runahead. In this case, we define a memory-resident variable `resident_mem`² that stores the same value as index.

¹tmp = reloadbuffer[array[index] << 12]

²Memory-resident variable can be declared using the volatile keyword.

To test load instruction, we load data from `resident_mem` to index during runahead execution. If load instruction functions correctly, we will see the cache-hit signal when we decode the reloadbuffer afterwards.

For store instruction, we modify the reloadbuffer encoding process to `tmp = reloadbuffer[array[resident_mem] << 12]`. Additionally, store the value from index to `resident_mem`. Store instruction is considered successfully executed if and only if we observe the cache-hit signal after decoding the reloadbuffer; otherwise, we should only see the noise signal.

6.4.3 Branch instructions. Branch instructions, such as BL, B, CBZ, and RET, are typically used to control the flow of a program. We tested these four branch instructions by introducing a dependency between index and an add instruction. For instruction CBZ and B, we can place the dependent add into a branch. For BL and RET, we can place the dependent add instruction within the function call. Suppose those instructions, along with the add instruction, behave as expected. In that case, we should observe the output string shifted by one byte, similar to the behavior described in section 6.4.1.

6.4.4 System instructions. In ARMv8, there are typically two kinds of system instructions. One kind focuses on instruction execution, such as controlling memory access orders and implementing execution barriers in scenarios involving out-of-order execution. The other type deals with reading from and writing to system registers, which are used to configure the CPU or operating system.

We examined two instructions that are used to control instruction execution for testing purposes. The first instruction, DMB, acts as a memory barrier instruction, ensuring that all memory accesses³ before DMB are completed and visible; otherwise, subsequent memory operations are stalled. The second instruction, ISB serves as an instruction-level barrier, which guarantees that all instructions following ISB are not executed until the preceding instructions ISB have been committed. When we place either ISB or DMB within the Spectre-v1 gadget, we should observe that there is no signal in reloadbuffer. It occurs because the memory or instruction barrier prevents the reloadbuffer encoding operation from executing until the branch has been resolved.

System registers are usually accessible at EL1 (Exception Level 1) or higher privilege levels. In the ARMv8 ISA, SCTLR (System Control Register) is used to configure hardware features such as the MMU (Memory Management Unit), alignment checks, and endianness, etc. PMCCNTR_EL0 (Performance Counter Register) is accessible in EL0 only if it is enabled in kernel mode. We utilize the page fault gadget 2 to explore the behavior of reading the performance counter PMCCNTR_EL0 and system register SCTLR. Specifically, we read the system register in runahead mode and use the fetched value to encode the reloadbuffer. Finally, we check for the cache-hit signal by decoding reloadbuffer in the exception handler function.

6.4.5 Results and discussion. As presented in Table 1, most instructions are executed as expected, indicating that Denver 2 usually obeys the data and control flow during runahead execution. However, floating-point and vector operations discussed earlier in 6.3

³DMB with default options encompasses both load and store operations.

are not executed; it could be a deliberate strategy to enhance the throughput of the runahead process.

In addition to floating-point and vector operations, some system instructions, such as DMB and those involving reading from system registers, are not executed successfully. The DMB instruction may be bypassed to prioritize throughput and achieve more aggressive memory-level parallelism performance. Furthermore, system configuration units may be inaccessible during the whole runahead for reasons related to security and reliability.

Table 1: ARM ISA-level behavior in runahead

Ins	Type	Semantics	Result
ADD	Arithmetic	$rd = rn + op2$	✓
SUB	Arithmetic	$rd = rn - op2$	✓
MUL	Arithmetic	$rd = rn * rm$	✓
UDIV	Arithmetic	$rd = rn / rm$	✓
EOR	Logic	$rd = rn \oplus op2$	✓
AND	Logic	$rd = rn \oplus op2$	✓
LSL	Logic	$rd = rn \ll rm$	✓
LSR	Logic	$rd = rn \gg rm$	✓
ORR	Logic	$rd = rn op2$	✓
MOV	Logic	$rd = rn$	✓
LDR	Memory	$rd = [addr]$	✓
STR	Memory	$[addr] = rn$	✓
B	Branch	$pc = offset$	✓
BL	Branch	$x30 = pc + 4; pc = pc + offset$	✓
RET	Branch	$pc = [X_N]$	✓
ISB	System	<i>InstructionSyncBarrier</i>	✓
DMB	System	<i>DataMemoryBarrier</i>	✗
MRS1	System	$rd = SCTLR$	✗
MRS2	System	$rd = PMCCNTRELO$	✗
FP	Arithmetic	Floating point	✗
Vector	Arithmetic	Vector SIMD	✗

6.5 Pseudo-retirement

Runahead employs a technique known as pseudo-retirement to commit results, thereby conserving hardware resources. In the design of runahead for out-of-order CPUs, the completed oldest instruction is considered to be committed [31]. During the pseudo-retirement phase, the CPU will release all resources related to the instruction, and the retirement register alias table (RAT), which maintains the architectural state of the mapping between the physical register file and the architectural state, is updated.

6.5.1 Speculative dependency depth. The ARM Cortex-A53 CPU can speculate on a limited number of instructions. However, it cannot store its temporary results due to a constrained register renaming system and a restricted reorder buffer. Consequently, if an instruction depends on prior results, the CPU cannot execute it [24]. Notably, we find that the speculative behavior of the Denver 2 CPU is similar to that of the ARM Cortex-A53, which indicates that the Denver 2 CPU has a limited capability to manage long chains of dependent instruction bundles.

```

1  /*
2  * 1. Initialize multiple pointers
3  * 2. Link N pointers and array_length:
4  ptrN-> ... -> ptr2 -> ptr1 -> array_length
5  */
6  size_t *ptr1, **ptr2, ..., **...*ptr_N;
7  ptr1 = &array_length;
8  ptr2 = &ptr1;
9  ...
10 ptr_N = &ptr_N_1
11 if(index < **...*ptr_N){
12     /*
13      * Dependent instruction bundle
14      * all registers are 0
15     */
16     x2 = x2 xor x5;
17     x3 = x3 xor x2;
18     x4 = x4 xor x3;
19     x5 = x5 xor x4;
20     index = index + x5;
21     /*No signal*/
22     tmp = reloadbuffer[array[index] << 12];
23 }

```

Listing 5: Speculative dependency management

As illustrated in code snippet 5, by increasing the levels of pointer dereferencing, we delay the resolution of the conditional branch in Spectre-v1 on Denver 2, creating a larger speculative window. However, when testing a sequence of instructions with data dependencies—where each instruction relies on the result of the previous one—we find that only the first dependent instruction executes speculatively. Specifically, if we add index with x2, we will get a cache hit signal; if we add index with x3, x4, or x5, we do not observe a signal in reloadbuffer. The behavior suggests that while the delayed branch resolution creates a sufficiently large instruction window to execute all instructions, the processor does not speculatively execute deeper into the dependency chain.

We can conclude that the CPU is only able to store the temporal result of the first instruction during speculative execution. If subsequent instructions rely on results from a deeper instruction stream, they cannot be executed. Notably, if we change the eor operation to add, the CPU can effectively manage the long dependency chain. In contrast, other instructions such as div, mul, and sub support this conclusion. The Denver 2 CPU may contain specific optimizations for add operations, such as a very deep bypass queue for add.

6.5.2 Runahead dependency depth. During the reverse engineering process, we observed that the runahead technique effectively handles deep instruction dependencies. Notably, runahead can retain all results within the instruction window, which is approximately 500 instructions' temporal results. Compared with speculative execution, which only handles the result of the very first instruction, the CPU is able to manage many more temporal results during runahead.

In speculative execution, temporal instruction results are typically stored in a reorder buffer or a physical register file. However, the Denver architecture does not include a reorder buffer [3], nor register renaming. While some lightweight register renaming and speculative execution may be deployed in the Denver 2 CPU, it may not feature a huge reorder buffer or a complicated register

renaming component. We believe that runahead may pseudo-retire instructions in the physical register file directly, although it may not own a reorder buffer. The CPU checkpoints the physical register file at the entry of runahead; when the CPU returns to normal mode, it will recover the register file using the backup register value. Since only the checkpointed register file is visible to the programmer, pseudo-retired results do not affect the architectural state.

7 RUNAHEAD SECURITY ANALYSIS

7.1 Attack overview

In addition to investigating Spectre-v1, we examined whether runahead execution enables attackers to launch other types of Spectre attacks. Our findings revealed that speculative store bypassing and speculation with a return buffer are effective on the Denver CPU. Furthermore, because the CPU skips vector and FP operations during runahead execution, we can develop a variant of Spectre attacks. Finally, the large instruction window, combined with the runahead’s effective management of instruction dependencies, enhances the effectiveness of Spectre-type attacks.

7.2 Speculative store bypassing

7.2.1 Spectre-SSB. Modern CPU architecture employs a store buffer to improve instructions per cycle (IPC). When executing a store instruction, the value is initially placed in the store buffer speculatively, allowing subsequent load operations to retrieve the value from the store buffer rather than from the data cache. However, when the target address is not resolved, the CPU may mispredict that the target memory address of the following load operation differs from the prior store, leading the memory unit to mistakenly load stale data from memory speculatively.

7.2.2 Store bypassing in runahead. In the early design of runahead, store instruction does not write the data anywhere but only marks the corresponding address as invalid. However, if the destination is not resolved, the CPU will skip the store operation, leaving the stale data still valid [10]. While the modern design of runahead allows store instructions to buffer data temporarily, the CPU may still fail to mark the destination as invalid if the address is not resolved. This issue can cause store operations in runahead to get bypassed.

The core that triggers speculative store bypassing in runahead is forcing the target memory to be unresolved for a store instruction. We start by initializing a M-value-buffer along with a index, which we then flush. Following this, we execute the store operation $\text{buffer}[\text{index}] = N$. Accessing index results in a cache miss, triggering the CPU to enter runahead mode while the target address remains unresolved. For example, if index is 1, we then encode the reloadbuffer with $\text{tmp} = \text{reloadbuffer}[\text{buffer}[1] \ll 12]$, which leads to the access of stale data M in the reloadbuffer.

7.3 Speculative attack via return stack buffer

7.3.1 Spectre-RSB. Modern processors typically use a return stack buffer to quickly restore the stack after returning from a function call. Specifically, when a function returns, the CPU retrieves the return address and the function frame to restore the function frame. If the return address is temporarily unavailable, the CPU will speculatively jump to the return address stored in the return stack buffer

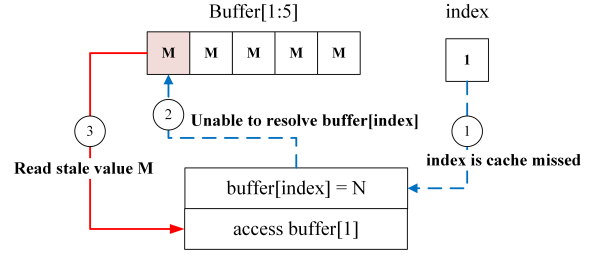


Figure 8: Spectre-SSB in runahead

instead of waiting for it to become available. In the Spectre-RSB attack, the attacker manipulates the stack by popping the return address and then flushing the stack pointer. When the function returns, because the return address is cache-missed, the CPU will instead return to the address in the return stack buffer, which points to the incorrect address. In the incorrect execution path, the attack can access sensitive data at the microarchitecture level [26].

7.3.2 Spectre-RSB via runahead. Denver 2 CPU also employs a return stack buffer to accelerate the function return process [7]. When we flush the stack pointer and trigger the CPU to enter runahead mode, because the CPU cannot retrieve the return address from the data cache, it will use the address from the return stack buffer. By preemptively popping the return address, we can trick the CPU into following an incorrect execution flow, allowing us to read sensitive data that is not accessible architecturally. Consequently, it is feasible to execute a microarchitectural attack using a standard Spectre-RSB gadget on a Denver 2 CPU operating in runahead mode.

7.4 Spectre-FPV-bypass

As discussed in the previous section 6.3, the CPU skips vector and FP operations during runahead. Based on this feature, we propose a variant of the Spectre-type attack. During runahead, if an operand of the branch is dependent on the result of a vector/FP operation, the branch instruction will be treated as invalid. By leveraging this characteristic, we can modify the Spectre-v1 gadget to facilitate the opening of an instruction window by adding vector or FP results as operands in the branch instruction.

In code snippet 6, we present the Spectre-FPV-bypass attack. Initially, we set a FP/vector register d0 to 0 and load cache-missed data to trigger runahead. Within the runahead instruction window, we introduce a dependency between `array_length` and `d0`. Since the CPU skips FP/vector operations during runahead, the branch will remain unresolved. If the CPU mispredicts the branch, it will execute instructions in the incorrect program flow, allowing the attacker to read sensitive data by performing a flush-and-reload attack.

8 EVALUATION

8.1 Spectre attacks

We tested all Spectre-typed attacks discussed in Section 7. Table 2 presents the results. For classic Spectre attacks, we evaluate Spectre-v1, Spectre-SSB, and Spectre-RSB, all of which perform effectively


```

1 d0 = 0;           // d0 is a FP/vector register
2 load cachemiss; //Trigger runahead
3 /*
4  *Introduce dependency between
5  *FP/vector register and array_length
6  */
7 array_length += d0;
8 if(index < array_length){
9     tmp = reloadbuffer[array[index] << 12];
10 }

```

Listing 6: A Spectre-v1 variant

in runahead execution. Besides classic Spectre attacks, we also assessed the new variant of the Spectre attack, Spectre-FPV-bypass, which leverages the skipped vector/FP operation. The new variant has also been proven effective in an attack.

Furthermore, we investigated if cross-privilege memory reading is possible in runahead. Specifically, we attempted to read the kernel module memory from user space using the Spectre-v1 gadget. Our finding indicates that the CPU does not permit cross-privilege reading, even during runahead. The MMU continues to perform memory permission checks during runahead. However, the memory permission check is not fully effective in runahead mode; for example, Spectre-v1.1 [23] allows us to write to read-only memory during runahead.

Table 2: Evaluation of Spectre attacks in runahead

Spectre attack	Result
Spectre-v1	✓
Spectre-v1.1	✓
Spectre-FPV-bypass	✓
Spectre-SSB	✓
Spectre-RSB	✓
Meltdown	✗

8.2 Potential end to end exploitation

Runahead enables the attacker to perform complicated microarchitectural attacks on the Denver 2 CPU because of the large effective instruction window. We believe that end-to-end exploitations are possible on the Denver 2 CPU.

8.2.1 Spectre attack. Since we have demonstrated that certain types of Spectre attacks are effective against the Denver 2 CPU, it should be possible to exploit real-world applications using Spectre attacks. To exploit real-world applications, it is needed to scan for Spectre gadgets. SpecTaint is a Spectre-v1 gadgets scanner based on taint analysis and has proven effective in finding gadgets in applications such as Caffe and Brotli [34]. InSpectre is an advanced framework specifically designed for scanning cross-privilege Spectre-v2 gadgets, and it includes techniques to analyze the details of potential exploitations automatically [44]. KASPER is another scanner that targets microarchitectural attack gadgets within the Linux kernel, revealing 1,379 unpatched vulnerable gadgets susceptible to

Spectre attacks [21]. With the scanners, it is feasible to identify potential Spectre gadgets that exist in both the kernel and user-space applications, which could allow an attacker to read sensitive data.

8.2.2 Spectre-FPV-bypass. Nvidia Jetson boards are designed for machine learning applications, particularly in computer vision. These applications typically rely heavily on vector operations for accelerated computation [8]. Additionally, cryptographic processes sometimes utilize vector operations to enhance speed [37]. For applications dependent on vector operations, it is possible to scan for potential gadgets similar to Spectre-FPV-bypass. We leave these gadgets scanning efforts for future work.

9 DISCUSSION

9.1 Limitations

Generally, runahead execution does not introduce entirely new gadgets; however, it is possible to exploit real-world applications and the kernel, similar to classic Spectre-type attacks. Additionally, while we propose Spectre-FPV-bypass as a new variant of Spectre gadgets, it is challenging to find this type of gadget in the kernel because the kernel normally prohibits using FP/vector operations [22], and successful exploitation highly depends on whether the attacks can identify exploitable gadgets in user-space applications. Furthermore, runahead is not deployed in most CPUs; only Nvidia Denver and IBM Power Series officially state that runahead is implemented in their products. These CPUs are generally used as accelerators, in contrast to the more commonly used desktop and server CPUs.

9.2 Future work

The Denver 2 CPU was publicly available in 2016; which means our analysis may only cover an older version of runahead. There are several papers discussing runahead and offering more advanced insights. For example, precise runahead execution focuses on pre-executing the instructions that are more likely to stall the pipeline and introduces a more advanced register renaming mechanism to release hardware resources quickly [32], which could result in a significantly larger transient attack window.

It is also possible that other kinds of microarchitectural optimizations have evolved from runahead, and these new optimizations could be more complicated and vulnerable. Since we have proposed a method to distinguish runahead and speculative execution 6.2, although most manufacturers do not officially state that runahead or similar optimizations are implemented in their CPUs. Thus, we can also check if a CPU exhibits behaviors similar to those of runahead execution.

10 RELATED WORK

Research on the security analysis of runahead execution is relatively limited. Most studies focus on techniques to improve the performance of runahead. However, many researchers employ CPU reverse engineering methods to identify security vulnerabilities related to microarchitectural security. Our research primarily focuses on CPU reverse engineering and security analysis, with a specific emphasis on runahead execution in the Denver architecture.

The early design of runahead concentrated on pre-executing instructions and prefetching data to warm the cache; however, store instructions, along with their dependent instructions, are skipped [10]. Over the years, researchers have begun to investigate the implementation of runahead execution in out-of-order CPUs and proposed a runahead cache to store the temporal results of store instructions [31]. Subsequently, researchers noticed that runahead consumes resources but does not bring significant performance benefits. As a result, they proposed efficient strategies for mitigation [30]. One such strategy involves skipping instructions, such as FP and vector, that are usually not used in address computation.

Additionally, some researchers have investigated pre-executing instructions in a separate thread to minimize resource contention [36]. To further improve throughput in runahead, an implementation that tracks memory load operations was proposed, which ensures the CPU only executes load instructions alongside their dependent instructions [15]. Finally, precise runahead execution specifically pre-executes instructions that cause the pipeline to stall and employs an advanced register renaming system to release the hardware resources promptly [32].

There is one notable research paper focusing on the security analysis of runahead execution, where the authors evaluate speculative attacks via runahead using the Multi2sim simulator, and conclude that runahead introduces more challenges to recent countermeasures against microarchitectural attacks [38]. We assess runahead in a commercially released CPU using a black-box test method, which uncovers security risks in a real-world context, which distinguishes our work from previous research.

11 CONCLUSION

The runahead feature in the NVIDIA Denver architecture shares similarities with speculative execution but also has unique characteristics. It allows the CPU to skip operations such as floating-point (FP) and vector calculations, which typically do not contribute to memory address computation. In a superscalar, in-order CPU, even if the processor struggles to manage temporal results during speculative execution, it can effectively handle complex data dependencies in runahead mode, which could make in-order CPUs more vulnerable to Spectre attacks. From a security perspective, runahead poses risks similar to Spectre attacks while increasing the vulnerability of in-order CPUs due to the pseudo-retirement of instructions. Additionally, the CPU's ability to skip vector operations during runahead introduces new invariants related to Spectre attacks, potentially creating a new attack surface.

REFERENCES

- [1] Nael Abu-Ghazaleh, Dmitry Ponomarev, and Dmitry Evtushkin. 2019. How the spectre and meltdown hacks really worked. *IEEE Spectrum* 56, 3 (2019), 42–49.
- [2] Ayush Agarwal, Sioli O'Connell, Jason Kim, Shaked Yehezkel, Daniel Genkin, Eyal Ronen, and Yuval Yarom. 2022. Spook.js: Attacking Chrome strict site isolation via speculative execution. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 699–715.
- [3] Darrell Boggs, Gary Brown, Bill Rozas, Nathan Tuck, and KS Venkatraman. 2014. Hot Chips 2014 Nvidia's denver processor. In *2014 IEEE Hot Chips 26 Symposium (HCS)*. IEEE, 1–25.
- [4] Darrell Boggs, Gary Brown, Nathan Tuck, and KS Venkatraman. 2015. Denver: Nvidia's first 64-bit ARM processor. *IEEE Micro* 35, 2 (2015), 46–55.
- [5] Harold W Cain and Priya Nagpurkar. 2010. Runahead execution vs. conventional data prefetching in the IBM POWER6 microprocessor. In *2010 IEEE International Symposium on Performance Analysis of Systems & Software (ISPASS)*. IEEE, 203–212.
- [6] Boru Chen, Yingchen Wang, Pradyumna Shome, Christopher Fletcher, David Kohlbrenner, Riccardo Paccagnella, and Daniel Genkin. 2024. {GoFetch}: Breaking {Constant-Time} Cryptographic Implementations Using Data {Memory-Dependent} Prefetchers. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1117–1134.
- [7] NVIDIA Corporation. 2014. *NVIDIA Jetson TX2 / TX2i System-on-Module Pascal GPU + ARMv8 + 8GB LPDDR4 + 32GB eMMC*. NVIDIA. Reference manual, https://developer.download.nvidia.com/assets/embedded/secure/jetson/tx2_nx/Jetson-TX2-NX-DS-10182-001_v1.5.pdf?_token__exp=1750058354-hmac=b3c11786602f565c564277a787626e716c78daab6bdcbb813e5b41b84981645&t=eyJscyl6lmdzZW8iLCJsc2QiOiJodHRwc2ovL3d3dy5nb29nbGUuY29tLjY9.
- [8] Robert Cypher and Jorge LC Sanz. 2002. SIMD architectures and algorithms for image processing and computer vision. *IEEE Transactions on Acoustics, Speech, and Signal Processing* 37, 12 (2002), 2158–2174.
- [9] Jack W Davidson and Sanjay Jinturkar. 1995. *An aggressive approach to loop unrolling*. Technical Report. Citeseer.
- [10] James Dundas and Trevor Mudge. 1997. Improving data cache performance by pre-executing instructions under a cache miss. In *Proceedings of the 11th international conference on Supercomputing*. 68–75.
- [11] Kemal Ebcioglu, Erik Altman, Michael Gschwind, and Sumedh Sathaye. 2001. Dynamic binary translation and optimization. *IEEE Transactions on computers* 50, 6 (2001), 529–548.
- [12] Stijn Eyerman, James E Smith, and Lieven Eeckhout. 2006. Characterizing the branch misprediction penalty. In *2006 IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE, 48–58.
- [13] Freddy Gabbay and Avi Mendelson. 1996. Speculative execution based on value prediction. (1996).
- [14] Siddharth Gupta, Yuanlong Li, Qingxuan Kang, Abhishek Bhattacharjee, Babak Falsafi, Yunho Oh, and Mathias Payer. 2023. Imprecise Store Exceptions. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.
- [15] Milad Hashemi, Onur Mutlu, and Yale N Patt. 2016. Continuous runahead: Transparent hardware acceleration for memory intensive workloads. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–12.
- [16] Milad Hashemi and Yale N Patt. 2015. Filtered runahead execution with a runahead buffer. In *Proceedings of the 48th International Symposium on Microarchitecture*. 358–369.
- [17] Joseph E Hollingsworth and Bruce W Weide. 1995. Micro-Architecture vs. Macro-Architecture. In *Proceedings of the Seventh Annual Workshop on Software Reuse*. 101–130.
- [18] Urs Hölzle, Craig Chambers, and David Ungar. 1992. Debugging optimized code with dynamic deoptimization. In *Proceedings of the ACM SIGPLAN 1992 conference on Programming language design and implementation*. 32–43.
- [19] CD James, KG Brian, PB John, J Richard, K Thomas, K Alexander, and M Jim. 2003. The transmeta code morphing™ software: Using speculation, recovery, and adaptive retranslation to address real-life challenges. In *Proceedings of the 1st International Symposium on Code Generation and Optimization*. San Francisco, California, USA, 15–24.
- [20] Hyerean Jang, Taehun Kim, and Youngjoo Shin. 2024. Sysbumps: Exploiting speculative execution in system calls for breaking kaslr in macos for apple silicon. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 64–78.
- [21] Brian Johannesmeyer, Jakob Koschel, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2022. Kasper: Scanning for Generalized Transient Execution Gadgets in the Linux Kernel. In *NDSS*, Vol. 1. 12.
- [22] The kernel development community. 2025. The Linux Kernel documentation. <https://docs.kernel.org/core-api/floating-point.html>. Accessed: 2025-07-11.
- [23] Vladimir Kiriansky and Carl Waldspurger. 2018. Speculative buffer overflows: Attacks and defenses. *arXiv preprint arXiv:1807.03757* (2018).
- [24] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre Attacks: Exploiting Speculative Execution. In *40th IEEE Symposium on Security and Privacy (S&P'19)*.
- [25] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, et al. 2020. Spectre attacks: Exploiting speculative execution. *Commun. ACM* 63, 7 (2020), 93–101.
- [26] Esmail Mohammadian Koruyeh, Khaled N Khasawneh, Chengyu Song, and Nael Abu-Ghazaleh. 2018. Spectre returns! speculation attacks using the return stack buffer. In *12th USENIX Workshop on Offensive Technologies (WOOT 18)*.
- [27] Gurhan Kucuk, Dmitry V Ponomarev, Oguz Ergin, and Kanad Ghose. 2004. Complexity-effective reorder buffer designs for superscalar processors. *IEEE Trans. Comput.* 53, 6 (2004), 653–665.
- [28] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval

- Yarom, and Mike Hamburg. 2018. Meltdown: Reading Kernel Memory from User Space. In *27th USENIX Security Symposium (USENIX Security 18)*.
- [29] Onur Mutlu, Hyesoon Kim, and Yale N Patt. 2005. Techniques for efficient processing in runahead execution engines. In *32nd International Symposium on Computer Architecture (ISCA'05)*. IEEE, 370–381.
- [30] Onur Mutlu, Hyesoon Kim, and Yale N Patt. 2006. Efficient runahead execution: Power-efficient memory latency tolerance. *IEEE Micro* 26, 1 (2006), 10–20.
- [31] Onur Mutlu, Jared Stark, Chris Wilkerson, and Yale N Patt. 2003. Runahead execution: An alternative to very large instruction windows for out-of-order processors. In *The Ninth International Symposium on High-Performance Computer Architecture, 2003. HPCA-9 2003. Proceedings*. IEEE, 129–140.
- [32] Ajeya Naithani, Josué Feliu, Almutaz Adileh, and Lieven Eeckhout. 2020. Precise runahead execution. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 397–410.
- [33] Hamed Nemati, Roberto Guanciale, Pablo Buiras, and Andreas Lindner. 2020. Speculative leakage in ARM cortex-a53. *arXiv preprint arXiv:2007.06865* (2020).
- [34] Zhenxiao Qi, Qian Feng, Yueqiang Cheng, Mengjia Yan, Peng Li, Heng Yin, and Tao Wei. 2021. SpecTaint: Speculative Taint Analysis for Discovering Spectre Gadgets. In *NDSS*. 1–14.
- [35] Tanausú Ramírez, Alex Pajuelo, Oliverio Jesus Santana, Onur Mutlu, and Mateo Valero. 2010. Efficient runahead threads. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*. 443–452.
- [36] Tanausú Ramírez, Alex Pajuelo, Oliverio J Santana, and Mateo Valero. 2008. Runahead threads to improve SMT performance. In *2008 IEEE 14th International Symposium on High Performance Computer Architecture*. IEEE, 149–158.
- [37] Binanda Sengupta and Abhijit Das. 2017. Use of SIMD-based data parallelism to speed up sieving in integer-factoring algorithms. *Appl. Math. Comput.* 293 (2017), 204–217.
- [38] Chaoqun Shen, Gang Qu, and Jiliang Zhang. 2025. Nested Speculative Execution Attacks via Runahead. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2025).
- [39] James E Smith. 1998. A study of branch prediction strategies. In *25 years of the international symposia on Computer architecture (selected papers)*. 202–215.
- [40] James E Smith and Gurindar S Sohi. 2002. The microarchitecture of superscalar processors. *Proc. IEEE* 83, 12 (2002), 1609–1624.
- [41] Robert M Tomasulo. 1967. An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of research and Development* 11, 1 (1967), 25–33.
- [42] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F Wenisch, Yuval Yarom, and Raoul Strackx. 2018. Foreshadow: Extracting the keys to the intel {SGX} kingdom with transient {Out-of-Order} execution. In *27th USENIX Security Symposium (USENIX Security 18)*. 991–1008.
- [43] Steven P Vanderwiel and David J Lilja. 2000. Data prefetch mechanisms. *ACM Computing Surveys (CSUR)* 32, 2 (2000), 174–199.
- [44] Sander Wiebing, Alvise de Faveri Tron, Herbert Bos, and Cristiano Giuffrida. 2024. {InSpectre} Gadget: Inspecting the Residual Attack Surface of Cross-privilege Spectre v2. In *33rd USENIX Security Symposium (USENIX Security 24)*. 577–594.
- [45] Henry Wong. 2013. Measuring Reorder Buffer (ROB) Capacity. <https://blog.stuffedcow.net/2013/05/measuring-rob-capacity/>. Accessed: 2025-07-08.
- [46] Yuval Yarom and Katrina Falkner. 2014. {FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack. In *23rd USENIX security symposium (USENIX security 14)*. 719–732.

ARTIFACTS

A ARTIFACT APPENDIX

A.1 Abstract

The paper evaluates runahead execution in the NVIDIA Denver 2 CPU. We demonstrate that runahead introduces extra security risks to in-order CPUs. Specifically, the attacks are capable of launching Spectre attacks, including Spectre-v1, Spectre-SSB, and Spectre-RSB. Additionally, we verify that the CPU bypasses FP/vector operations during runahead execution. With this feature, we develop a new variant for Spectre gadgets.

A.2 Description & Requirements

A.2.1 Security, privacy, and ethical concerns. There are no security or privacy risks when executing the artifacts. All experiments are conducted locally, ensuring they do not involve end-to-end exploitation.

A.2.2 How to access. The source code can be accessed by <https://github.com/r3c0nb4/denver>.

A.2.3 Hardware dependencies. NVIDIA Denver 2 CPU is necessary for the artifacts. The CPU is integrated into the NVIDIA Jetson TX2 NX board. However, the board only contains the CPU and DRAM; a carrier or development kit is also needed to run programs with the Denver 2 CPU.

A.2.4 Software dependencies. Jetson Linux 32.7.6 is the newest version of the OS that can run on the Jetson TX2 NX board, and it is included in JetPack 4.6.6.

A.2.5 Benchmarks. None.

A.3 Set-up

A.3.1 Installation. Install the NVIDIA SDK Manager on Ubuntu 16.04 or 18.04, which can be downloaded from <https://developer.nvidia.com/sdk-manager>. Then use the SDK Manager to flash the Jetson Linux system onto the Jetson board. Run `enable_hugepage` script in the Denver folder to enable 64K page size. Load the kernel module in `denver/performance_counter/` to enable the performance counter access in user mode.

A.3.2 Basic Test. Connect the UART port to the computer, then connect to Jetson (192.168.55.1) with SSH, and we should be able to execute programs and write testing C programs on the Jetson board.

A.4 Evaluation workflow

- (C1): We demonstrate that the CPU bypasses floating-point (FP) operations during runahead. However, FP operations are executed as usual in speculative execution, indicating that bypassing FP operations is the distinctive feature in runahead. E1 presents the reproduction of the claim.
- (C2): Denver 2 has limited capability to manage instruction dependency in speculative execution. When we explore the Spectre-v1 with a long dependent instruction bundle within the instruction window, we do not observe a cache hit signal in the `reloadbuffer`. However, if we conduct the same

experiment in runahead, we can see the signal, indicating runahead employs techniques like pseudo-retirement to manage the instruction dependency effectively. The experiments can be reproduced by following procedures in E2.

- (C3): Apart from Spectre-v1, we also mount speculative store bypassing, Spectre-RSB, and a new variant, Spectre-FPV-bypass. Section 8 presents the results. E3 presents the procedures to reproduce the experiment.

A.4.1 Experiments.

- (E1): [1 compute-minute]: The experiment runs Spectre-v1 via speculative execution and runahead separately, in order to prove the FP operations are bypassed in runahead, but executed as usual in Speculation.

Preparation: Navigate to folder `denver/runahead/fingerprint`

Execution: Run script `spectre.sh` in folder `speculative/` and `runahead/` separately.

Results: In speculative execution, we shall see a cache-hit signal: `SPECTRE_EXECUTE`, indicating FP operations are executed as usual in speculative execution; however, in the other case, when triggering runahead, we shall not see any signals, just 0-valued characters in the output.

- (E2): [1 compute-minute]: The experiment is used to prove that Denver 2 lack capability of managing instruction bundles with strong dependency. We employ Spectre-v1 gadget in speculative execution and runahead separately to compare the results. In both of the two minor experiments, we place a few instructions before the `reloadbuffer` encoding process, and all instructions depend on the result of previous one.

Preparation: Navigate to folder `denver/runahead/spectre-dependency`

Execution: Run script `spectre.sh` in `runahead/` and `speculative/` folder separately.

Results: In speculative execution, we shall not see any cache-hit signals. However, if we modify the arm assembly code in Spectre-v1 gadget, and let index depend on x2, we will see the signal. It indicates that the speculative window is large enough, but the data dependency chain is not preserved. However, in runahead execution, we shall see cache hit signal: `SPECTRE_EXECUTE`.

- (E3): [1 compute-minute]: Test Spectre-FPV-bypass, Spectre-SSB and Spectre-RSB in runahead.

Preparation: Navigate to folder `denver/runahead/spectre-attacks`

Execution: Run `spectre.sh` in folder `spectreRSB`, `spectre-FPV-bypass`, and `SSB` separately.

Results: For Spectre-FPV-bypass and Spectre-RSB, we shall see cache hit signal: `SPECTRE_EXECUTE`. In Spectre-SSB, we shall observe stale value `0x1` in the `reloadbuffer`.

A.5 Version

Based on the LaTeX template for Artifact Evaluation V20231005. Submission, reviewing and badging methodology followed for the evaluation of this artifact can be found at <https://secartifacts.github.io/usenixsec2024/>.